在 RoPE（旋转位置编码）中：

$$\theta = pos \cdot \omega_i$$

这里的 pos 和 ω_i 分别是 Token 的位置和第 i 组维度对应的旋转频率。

1. pos：Token 在序列中的位置

pos = position，表示当前 Token 是第几个。

例如输入：

我　喜欢　学习　大模型

假设每个词是一个 Token：

Token	pos
我	0
喜欢	1
学习	2
大模型	3

因此：

第一个 Token：`pos = 0`

第二个 Token：`pos = 1`

第三个 Token：`pos = 2`

在实际 Qwen 中，文本会先经过 Tokenizer，位置通常对应 Token 序列下标，而不是中文词语的下标。

pos 越大，表示 Token 在序列中越靠后。

2. ω_i：第 i 组维度的旋转频率

ω_i 读作 omega 下标 i。

它表示：第 i 组二维向量，每向前移动一个位置，需要旋转多快。

经典 RoPE 的频率定义为：

$$\omega_i = 10000^{-2i/d}$$

其中：

i：第几组二维维度，从 0 开始。

d：Attention Head 的维度。

10000：经典 RoPE 的基数。

注意：不同模型的 RoPE 频率配置可能不同，不能把所有 Qwen 版本都直接套用这一公式。

举个例子

假设一个 Attention Head 的维度：

d=8

那么它有 4 组二维维度：

i	维度对	ω_i
0	(0,1)	1
1	(2,3)	0.1
2	(4,5)	0.01
3	(6,7)	0.001

计算方式：

$$\omega_0 = 10000^{-0/8} = 1$$

$$\omega_1 = 10000^{-2/8} = 0.1$$

$$\omega_2 = 10000^{-4/8} = 0.01$$

$$\omega_3 = 10000^{-6/8} = 0.001$$

前面的维度旋转快，后面的维度旋转慢。

3. pos × ω_i：真正的旋转角度

RoPE 的核心是：

$$\theta_{pos,i} = pos \cdot \omega_i$$

例如：

Token 位置 `pos = 10`

第 0 组频率 $\omega_0 = 1$

那么：

$$\theta_{10,0} = 10 \times 1 = 10$$

也就是旋转 10 弧度。

再看第 2 组：

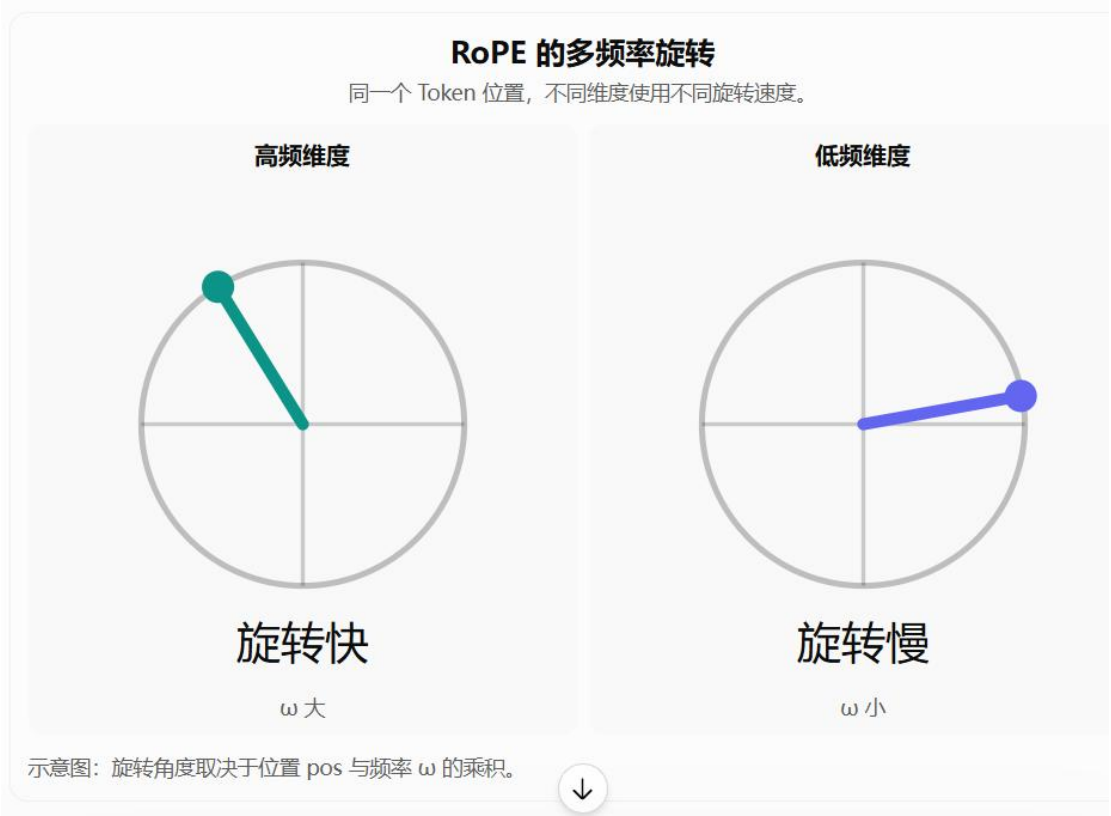
$$\theta_{10,2} = 10 \times 0.01 = 0.1$$

也就是旋转 0.1 弧度。

同一个 Token，在不同维度组上会有不同的旋转角度。

4. 直观理解：快时钟与慢时钟

把每一组二维向量想成一个时钟：



这样设计的好处是：

高频维度：对相邻 Token 的位置变化敏感。

低频维度：变化缓慢，可以表示更长距离的位置关系。

5. 为什么要乘以 pos？

假设某一组频率：

$\omega_i = 0.1$

那么不同 Token 的旋转角度是：

pos θ = pos × 0.1
0 0
1 0.1
2 0.2
10 1.0

20 2.0

每往后移动一个 Token，旋转角度增加：

$\Delta\theta = \omega i$

所以位置编码本质上是在告诉模型：

> 这个 Token 位于序列的什么位置，以及它与其他 Token 的相对位置关系。

6. 用 PyTorch 看实际计算

```
import torch
# 一个 attention head 的维度 d = 8
# Token 位置 pos = torch.tensor([0, 1, 2, 3])
# 4 组二维维度的频率 i = torch.arange(0, d, 2)
omega = 10000 ** (-i / d)
print("omega:", omega)
# 计算不同 Token 位置的旋转角度 theta = pos[:, None] * omega[None, :]
print("theta:")print(theta)
```

输出的频率大致是：

```
tensor([1.0000, 0.1000, 0.0100, 0.0010])
```

角度矩阵：

```
pos=0: [0.000, 0.000, 0.000, 0.000]
pos=1: [1.000, 0.100, 0.010, 0.001]
pos=2: [2.000, 0.200, 0.020, 0.002]
pos=3: [3.000, 0.300, 0.030, 0.003]
```

其中每一列对应一个二维维度组。

7. 最后把它和 Qwen Attention 联系起来

Qwen2.5 的 Attention 中，RoPE 大致是：

```
# hidden_states: [batch, seq_len, hidden_size]
Q = self.q_proj(hidden_states)K = self.k_proj(hidden_states)V =
self.v_proj(hidden_states)
# 将 Q、K reshape 成多头 Q = Q.view(batch, seq_len, num_heads,
head_dim)K = K.view(batch, seq_len, num_kv_heads, head_dim)
# 生成每个位置的 cos/sincos, sin = rotary_emb(Q, position_ids)
```

```
# 对 Q、K 旋转 Q = apply_rotary_pos_emb(Q, cos, sin)K =
apply_rotary_pos_emb(K, cos, sin)
# 然后进行 GQA / Attention
```

其中：

`position_ids`：每个 Token 的位置 `pos`。

`rotary_emb`：根据位置生成旋转角度。

`cos` / `sin`：旋转矩阵需要的三角函数值。

`apply_rotary_pos_emb`：真正把旋转应用到 Q 和 K。

一句话记忆：

$\theta = pos \cdot \omega_i$

`pos` 决定“当前 Token 在哪里”，ω_i 决定“这一组维度转多快”，两者相乘得到“该维度组应该旋转多少角度”。

传统 transformer 的位置编码和 qwen 的 RoPE 的区别

可以把它理解成一句话：

> **传统 Transformer 的位置编码是“把位置加到 token 向量上”；Qwen 的 RoPE 是“把位置信息直接旋转到 Attention 的 Q、K 里面”。**

而且这里有一个很重要的点：**RoPE 并不是完全不同的数学思想，它和 Transformer 最初的 Sinusoidal Position Encoding 有很深的关系。**

---

1. 先看传统 Transformer

原始 Transformer（2017）使用的是 **Sinusoidal Positional Encoding**。

假设：

```
Token embedding:
x₁  = "我"
x₂  = "喜欢"
x₃  = "AI"
```

Transformer 本身只看到三个向量：

```
x₁
x₂
x₃
```

但 Attention 本身不知道：

```
我 → 喜欢 → AI
```

和：

```
AI → 喜欢 → 我
```

有什么区别。

所以需要告诉模型：

> “你现在处理的是第几个 token。”

---

2. 传统方法：直接加位置向量

假设 token embedding 是：

x_1

位置编码是：

PE_1

那么 Transformer 输入：

h_1=x_1+PE_1

第二个 token：

h_2=x_2+PE_2

第三个：

h_3=x_3+PE_3

也就是：

```
token embedding
      +
position embedding
      ↓
Transformer
```

这是原始 Transformer 的核心做法。

---

3. Sinusoidal PE 到底是什么？

它不是简单地：

```
位置 1 → [1,0,0,0]
位置 2 → [2,0,0,0]
位置 3 → [3,0,0,0]
```

而是使用不同频率的 sin/cos：

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

比如一个很低维的例子：

```
position 0 → [sin(0), cos(0), sin(0/100), cos(0/100)]
position 1 → [sin(1), cos(1), sin(1/100), cos(1/100)]
position 2 → [sin(2), cos(2), sin(2/100), cos(2/100)]
position 3 → [sin(3), cos(3), sin(3/100), cos(3/100)]
```

所以可以把它想象成：

```
第 0 个位置 → 一组“钟表指针”
第 1 个位置 → 指针转了一点
第 2 个位置 → 又转了一点
第 100 个位置 → 转得更多
```

不同维度使用不同频率。

---

4. 然后进入 Qwen 的 RoPE

Qwen 使用的是 **RoPE（Rotary Position Embedding）**。Qwen 当前实现中，会计算每个 position 对应的 `cos/sin`，然后直接作用到 Query 和 Key 上。

传统 Transformer：

```
Token
  ↓
Embedding + Position Encoding
  ↓
Q K V
  ↓
Attention
```

RoPE：

```
Token
  ↓
Embedding
  ↓
Q K V
  ↓
Q、K 做旋转
  ↓
```

Attention

这是两者最核心的区别。

---

5. 为什么叫 Rotary？

因为它真的就是“旋转”。

先把问题简化成二维。

假设：

$$q = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$$

位置为 m。

RoPE 对它进行旋转：

$$q'_m = R(m\theta)q$$

其中：

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

所以：

```
原来的 q
    ↘
      ↘ 旋转 mθ
        ↘
         q'
```

Key 也一样：

$$k'_n = R(n\theta)k$$

---

6. 最关键的地方来了

Attention 最终要计算：

$$q^T k$$

RoPE 后：

$$(q'_m)^T k'_n$$

代进去：

$$(R(m\theta)q)^T (R(n\theta)k)$$

可以得到：

$$q^T R((n-m)\theta)k$$

注意这里出现了：

n-m

也就是：

两个 token 之间的相对距离。

这就是 RoPE 最漂亮的地方。

---

7. 举个非常直观的例子

假设：

```
我     喜欢     AI
0       1       2
```

考虑：

```
我 → AI
```

它们的位置：

```
m = 0
n = 2
```

RoPE 最终 Attention 中体现的是：

$n-m=2$

换成：

```
今天    我    喜欢    AI
 0      1      2      3
```

现在：

```
我 → AI
```

位置变成：

```
m = 1
n = 3
```

但是：

$n-m=2n-m=2$

距离还是 2。

所以 RoPE 天然能够让 Attention 感受到：

> “这两个 token 相隔 2 个位置。”

而不是只能知道：

> “这个 token 是第 1 个，那个 token 是第 3 个。”

---

8. 传统 PE 和 RoPE 最大区别

可以直接看这个表：

	传统 Sinusoidal PE	**Qwen RoPE**
位置怎么加入	加到 embedding	旋转 Q/K
作用位置	Transformer 输入	Attention 内部
位置性质	绝对位置为主	相对距离自然出现
是否修改 Q/K	❌	✅
是否修改 V	❌	通常 ❌
是否有额外可学习参数	❌	❌

	传统 Sinusoidal PE	Qwen RoPE
Attention 是否直接感知相对位置	较间接	✓
长上下文扩展	相对麻烦	更容易做各种 scaling
Qwen	✗	✓

原始 Transformer 的位置编码是加法式的，而 RoPE 是作用于 Q/K 的旋转式位置编码。

---

9. 其实两者还有一个非常有意思的关系

你可能会发现：

RoPE 的 sin/cos 和传统 Sinusoidal PE 长得非常像。

这不是巧合。

传统 PE：

$$\sin(\text{pos}\,\theta)$$
$$\cos(\text{pos}\,\theta)$$

RoPE：

$$R(\text{pos}\,\theta)$$

也就是：

```
传统 PE
    ↓
sin / cos 产生一个“位置向量”

RoPE
    ↓
sin / cos 产生一个“旋转角度”
    ↓
旋转 Q/K
```

所以可以粗略理解为：

> **RoPE 把“Sin/Cos 位置编码”的思想，从“构造一个位置向量”升级成了“用位置控制 Q/K 的旋转”。**

10. 为什么 RoPE 通常比直接相加更适合 LLM？

这是非常关键的一点。

传统方法：

h_i=x_i+PE_i

然后：

$$Q = h_i W_Q$$

$$K = h_i W_K$$

Attention：

QK^T

于是里面实际上会出现很多混合项。

例如：

$$(x_i + PE_i)^T (x_j + PE_j)$$

展开：

$$x_i^T x_j + x_i^T PE_j + PE_i^T x_j + PE_i^T PE_j$$

也就是说：

内容
+
位置
↓
混在一起

模型自己需要学会：

哪些是内容关系，哪些是位置关系。

---

11. RoPE 则更加“直接”

RoPE：

$$q_i' = R_i q_i$$

$$k_j' = R_j k_j$$

然后：

$$(q_i')^T k_j'$$

最终变成：

$$q_i^T R_{j-i} k_j$$

所以：

```
内容
 +
相对位置
 ↓
Attention score
```

而且相对位置是通过数学结构直接产生的。

---

12. Qwen 里面到底怎么实现？

简化之后就是：

```
q = Wq(x)
k = Wk(x)
v = Wv(x)
q = RoPE(q, position)
k = RoPE(k, position)
```

```
attention = softmax(q @ k.T / sqrt(d))
output = attention @ v
```

Qwen 的实现也是对 `query_states` 和 `key_states` 应用：

```
q_embed = q * cos + rotate_half(q) * sin
k_embed = k * cos + rotate_half(k) * sin
```

然后再进行正常 Attention。

这里：

```
rotate_half(q)
```

本质就是把二维向量：

```
[x1, x2]
```

变成：

```
[-x2, x1]
```

再配合：

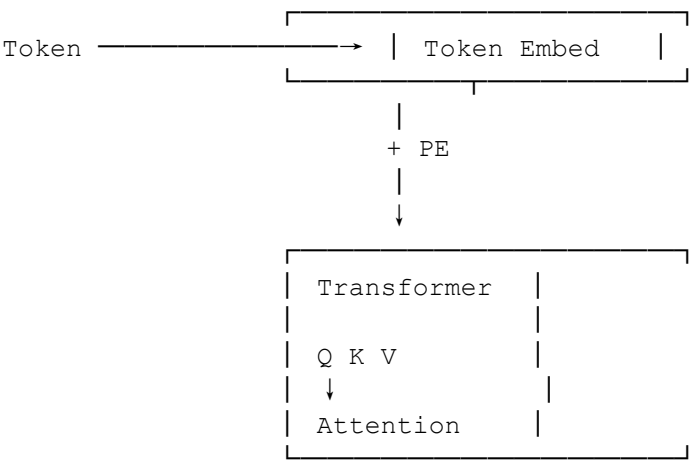
```
cos
sin
```

实现二维旋转。

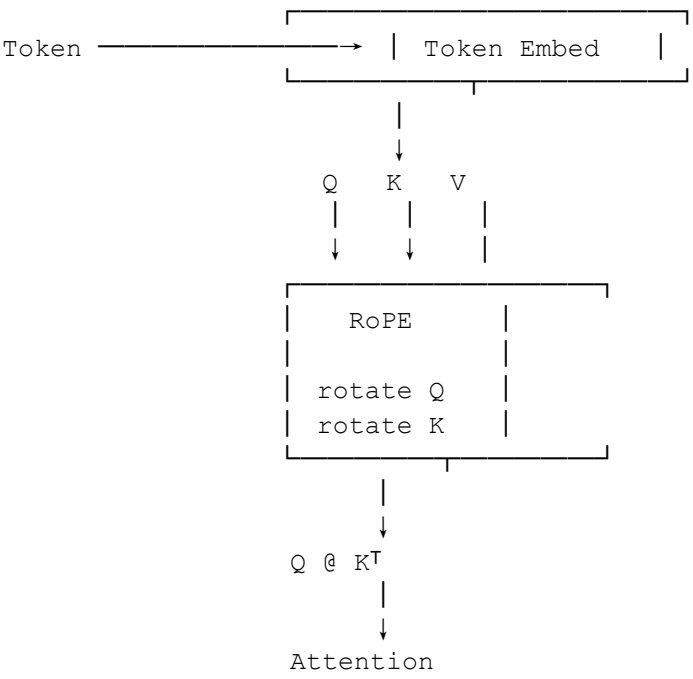
---

13. 用一张流程图理解最清楚

传统 Transformer



Qwen / RoPE



14. 还有一个容易搞错的地方

RoPE 不是 Qwen 自己发明的。

RoPE 来自 RoFormer / Rotary Position Embedding 思路，后来被大量现代 LLM 采用，包括 Llama、Qwen、DeepSeek 等。

所以：

```
Transformer
    ↓
Sinusoidal PE
    ↓
各种位置编码改进
    ↓
RoPE
    ↓
Llama / Qwen / DeepSeek / Mistral ...
```

15. 你可以用一句话彻底区分

如果面试问你：

> 传统 Transformer 的位置编码和 Qwen 的 RoPE 有什么区别？

你可以直接回答：

> 原始 Transformer 的 Sinusoidal Position Encoding 是把表示“第几个位置”的向量直接加到 token embedding 上，属于输入层的绝对位置编码；Qwen 使用 RoPE，则是在 Attention 中对 Query 和 Key 按 token 的位置进行旋转，使 Q·K 的计算天然包含相对位置信息，因此不需要把位置向量直接加到 token embedding 上。

再进一步：

> Sinusoidal PE 是“位置 → 向量”；RoPE 是“位置 → 旋转角度 → 旋转 Q/K”。

这个理解基本就到位了。

如果你接下来想真正搞懂 Qwen，我建议下一步直接看 **“RoPE 为什么能让 QKTQK^T 得到相对位置 m−nm-n”的完整矩阵推导**。这个推导其实非常漂亮，而且理解之后你会自然明白 **RoPE、NTK scaling、YaRN、长上下文扩展**之间是什么关系。